🚀 GUÍA TÉCNICA PROFESIONAL

Solución 2: Delegación de Eventos

✍️ Por Ramón Abramo • Profesor de Desarrollo Web

¡Buenas! Soy Ramón Abramo. Bienvenidos a la guía de la **Solución 2**, el patrón que utilizan los desarrolladores profesionales en aplicaciones reales. Aquí damos un salto de calidad: en lugar de registrar 16 escuchadores en cada tecla, registraremos **un único escuchador en el contenedor padre** `.teclado` y aprovecharemos el borboteo de eventos (*Event Bubbling*).

🎯 1. La Filosofía de la Delegación

A mis alumnos siempre les pongo la analogía del conserje: *"Si tenéis un edificio con 16 despachos, ¿contratáis a 16 guardias de seguridad para cada puerta o ponéis a un solo vigilante experto en la entrada principal?"* La respuesta es evidente.

🌊 1. El Borboteo

Cualquier click en un botón viaja hacia arriba por el DOM hasta alcanzar al padre `.teclado`.

🎯 2. El Interruptor

Con `e.target.closest('button')` averiguamos exactamente qué tecla pulsó el usuario.

⚡ 3. Cero Fugas

Un solo listener en memoria; máxima eficiencia y escalabilidad ante botones dinámicos.

📂 Archivos de esta Carpeta (`version-2-delegacion-eventos/`)

- `app.js` : Código completo con los 7 hitos y el despachador de eventos único.
- `app.alumno.js` : Plantilla con bloques `TODO` vacíos para el alumnado.
- `GUIA_PASO_A_PASO.md` : Documento de apoyo.
- Las diapositivas (`presentacion_solucion_2_delegacion.html`) están en la carpeta `documentacion/`.

🌊 Cómo Funciona el Borboteo (Event Bubbling)

> En el estándar de eventos del W3C, cuando el usuario interactúa con un elemento de la pantalla, el evento atraviesa tres fases: Captura, Objetivo y **Borboteo (Bubbling)**.

El Camino de un Click en la Calculadora

1. **Fase Objetivo:** El usuario pulsa con el ratón sobre el botón `<button data-numero="7">7</button>`.
2. **Fase de Borboteo:** Como una burbuja de aire en el agua, el evento sube a su elemento padre: `<div class="teclado">`.
3. **La Interceptación:** Nuestro escuchador colocado en `.teclado` detecta la burbuja y la captura inmediatamente.
4. **La Identificación:** Preguntamos al evento: *"¿Quién fue el elemento original donde se hizo click?"* La respuesta está en `evento.target`.

2. Selección Minimalista de Nodos

Fijaos en la enorme diferencia con la Solución 1. En lugar de seleccionar 5 grupos de botones, aquí **solo seleccionamos dos cosas**:

```
// Pantalla donde se reflejan los resultados
const display = document.getElementById('display');

// UN SOLO CONTENEDOR: El padre de todos los botones
const teclado = document.querySelector('.teclado');
```

> 💡 **Ventaja Inmediata**
>
> Reducimos drásticamente el acoplamiento entre nuestro archivo JavaScript y el HTML. Si mañana en el HTML cambiamos el orden de los botones o añadimos un botón nuevo, este código sigue intacto sin tener que retocar selectores.

🛠️ Los 3 Secretos Técnicos de la Delegación

Para que la delegación sea robusta y profesional en clase, debemos aplicar tres técnicas clave:

1. El método 'closest("button")' (Protección contra Clics Fantasma)

¿Qué ocurre si el alumno hace clic en el margen de 10px entre dos teclas? `evento.target` sería el propio `<div class="teclado">`. Si intentáramos leer `dataset` de ese div, fallaría.
Al usar `const boton = evento.target.closest('button');`, si el clic no ocurrió dentro de un botón, `boton` valdrá `null` y salimos con un simple `if (!boton) return;`.

2. Desestructuración Limpia de Dataset

En lugar de escribir líneas repetitivas, extraemos los tres posibles atributos con una sola línea de desestructuración:
`const { numero, operador, accion } = boton.dataset;`

3. Enrutamiento por Exclusión

Cada botón solo tiene uno de los tres atributos. Por ejemplo:
- Si pulsó `8` ➔ `numero === '8'`, los demás son `undefined`.
- Si pulsó `*` ➔ `operador === '*'`, los demás son `undefined`.
- Si pulsó `=` ➔ `accion === 'igual'`.

La Memoria Interna: Idéntica a la Solución 1

```
let operandoActual = '0';
let operandoAnterior = null;
let operacion = null;
let reiniciarDisplay = false;

function actualizarDisplay() {
    display.textContent = operandoActual;
}
```

Lógica de Negocio: Desacoplada e Impecable

> Fijaos en una lección crucial de ingeniería de software: las funciones `agregarNumero()`, `seleccionarOperador()`, `calcular()` y `borrar()` son **100% idénticas** a las de la Solución 1.

```
// [HITO 1] Agregar dígito
function agregarNumero(numero) {
    if (reiniciarDisplay) {
        operandoActual = numero;
        reiniciarDisplay = false;
    } else {
        operandoActual = (operandoActual === '0') ? numero : operandoActual + numero;
    }
    actualizarDisplay();
}

// [HITO 2 y 5] Selección de operador y encadenamiento
function seleccionarOperador(nuevoOperador) {
    if (operacion !== null && !reiniciarDisplay) {
        calcular();
    }
    operandoAnterior = operandoActual;
    operacion = nuevoOperador;
    reiniciarDisplay = true;
}

// [HITO 3, 4 y 7] Motor de cálculo con parseFloat y switch
function calcular() {
    if (operacion === null || operandoAnterior === null) return;
    const anterior = parseFloat(operandoAnterior);
    const actual = parseFloat(operandoActual);
    let resultado;

    switch (operacion) {
        case '+': resultado = anterior + actual; break;
        case '-': resultado = anterior - actual; break;
        case '*': resultado = anterior * actual; break;
        case '/':
            resultado = (actual === 0) ? 'Error' : anterior / actual;
            break;
        default: return;
    }

    if (typeof resultado === 'number') {
        resultado = Math.round(resultado * 100000000) / 100000000;
    }

    operandoActual = resultado.toString();
    operacion = null;
    operandoAnterior = null;
    reiniciarDisplay = true;
    actualizarDisplay();
}

// [HITO 6] Borrado total
function borrar() {
    operandoActual = '0';
    operandoAnterior = null;
    operacion = null;
    reiniciarDisplay = false;
    actualizarDisplay();
}
```

El Despachador de Eventos en Acción

> En este único bloque se condensa todo el control de eventos de la calculadora. Observad la elegancia y limpieza del código:

```
// UN SOLO ESCUCHADOR PARA TODAS LAS TECLAS
teclado.addEventListener('click', (evento) => {
    // 1. Interceptamos el botón clickeado
    const boton = evento.target.closest('button');
    if (!boton) return; // Ignoramos clics en el contenedor

    // 2. Extraemos los atributos data-*
    const { numero, operador, accion } = boton.dataset;

    // 3. Enrutamiento según el tipo de tecla:
    if (numero !== undefined) {
        // [HITO 1] Es un dígito (0 al 9)
        agregarNumero(numero);

    } else if (operador !== undefined) {
        // [HITO 2, 4 y 5] Es un operador (+, -, *, /)
        seleccionarOperador(operador);

    } else if (accion === 'igual') {
        // [HITO 3] Es la tecla =
        calcular();

    } else if (accion === 'borrar') {
        // [HITO 6] Es la tecla C
        borrar();
    }
});
```

🍼 Verificación con Console.log para Alumnos

En clase les pido que añadan un `console.log('Tecla detectada:', boton.dataset);` dentro del listener. Al abrir las DevTools (F12), ven en directo cómo cada clic imprime su objeto de datos antes de disparar la función correspondiente.

🚀 Por Qué Este Patrón Domina el Mercado

> Frameworks como React, Vue o Angular y librerías de alto rendimiento utilizan delegación de eventos internamente. Cuando vuestros alumnos dominen este concepto, estarán preparados para cualquier entorno profesional.

Eficiencia en Memoria

1 listener frente a 16. En aplicaciones web complejas con listas de cientos de filas o teclados dinámicos, la delegación evita fugas de memoria (*memory leaks*).

Tolerancia a Cambios Dinámicos

Si mediante JavaScript inyectamos un botón nuevo (por ejemplo una tecla `%` o `x²`), funcionará al instante sin tener que volver a vincular un escuchador.

✅ Batería de Validación de la Solución 2

Prueba	Secuencia de Clics	Resultado Esperado
Suma simple	`1 5 + 4 =`	`19`
Operación encadenada	`5 + 3 * 2 =`	`16` (8 * 2)
División por cero	`8 / 0 =`	`Error`
Reseteo C	`9 9 9 C`	`0`

¡Felicidades! Has completado el dominio del patrón de Delegación de Eventos. ▮ 🚀